

Internal Working of extend() in Python List

Before understanding this method

A Python list works like a dynamic array. Internally it has two important ideas:

- **size** - number of elements currently present in the list.
- **capacity** - extra internal space reserved by Python for future elements.

When capacity is full, Python creates a bigger internal array, copies old elements, and then performs the operation.

Meaning

extend() adds multiple elements from another iterable at the end of the list.

```
a = [10, 20, 30]
b = [40, 50]
a.extend(b)
print(a)    # [10, 20, 30, 40, 50]
```

Internal steps

- Python checks how many elements are coming from the iterable, when possible.
- Python checks whether the current list has enough capacity.
- If capacity is not enough, Python allocates bigger memory.
- New elements are copied/added one by one at the end.
- The list size is increased according to the number of added elements.

Before extend:

```
a = [10, 20, 30]
b = [40, 50]
```

Step 1: Add 40 -> [10, 20, 30, 40]

Step 2: Add 50 -> [10, 20, 30, 40, 50]

extend() is like repeated append()

```
a.extend([40, 50])
```

```
# Similar idea:
for item in [40, 50]:
    a.append(item)
```

append() vs extend()

```
a = [1, 2, 3]
a.append([4, 5])
print(a)    # [1, 2, 3, [4, 5]]
```

```
a = [1, 2, 3]
a.extend([4, 5])
print(a)    # [1, 2, 3, 4, 5]
```

Time complexity

- If k elements are added, time complexity is $O(k)$.
- If resizing happens, old elements may also be copied internally.

Key point

`extend()` adds elements separately at the end. It does not add the whole list as one element.